

Internet of Things Lab

(BCAP – 306)



Department of Computer Applications

Maharaja Surajmal Institute

C – 4, Janakpuri

New Delhi -110058

Submitted to:

Submitted by:

DR. Kumar Gaurav

(Associate Professor)

Name of student

BCA – 6 B (Morning)

Enrol. No:

List of Practicals

S.No.	Detailed Statement	Mapping to CO #
1.	Study and Install IDE of Arduino	CO1,CO2
2.	Write the steps to add libraries in Arduino and setup of Arduino IDE for programming.	CO2, CO3
3.	Write a Program using Arduino for Blink LED.	CO2, CO3
4.	Write a Program for monitoring Temperature using Arduino and LM35 Temperature Sensors.	CO2, CO3,
5.	Write a Program for Controlling Raspberry Pi with WhatsApp.	
6.	Write a program to shows how to fade an LED on pin 9 using the analog Write())function.	CO3,CO4
7.	Write the steps to add blynk libraries for NodeMCU and account on IFTTT for home automation.	CO2, CO3,CO4
8.	Write a program of Fade LED using NodeMCU(ESP8266) and blynk app	CO2, CO3,CO4
9.	Write a program for Arduino by using Ultrasonic sensors and servo motor (HC-SR04), and make a smart dustbin.	CO2, CO3,CO4
10.	Write a program for controlling bulb on/off by using Blynk app.	CO2, CO3,CO4

Q- Write a program for controlling bulb on/off using Blynk app.

```
#define BLYNK_PRINT Serial    // Comment this out to disable prints and save space
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <SPI.h>
#include <TimeLib.h>
#include <SimpleTimer.h>

// You should get Auth Token in the Blynk App.
// Go to the Project Settings (nut icon).
char auth[] = "token";

// Your WiFi credentials.
// Set password to "" for open networks.
char ssid[] = "wifi name";
char pass[] = "password";

const int btnPin1 = 1;        // pin for your physical push btn switch or toggle switch. ju
const int RelayPin1 = 3;      // pin for your relay outputs or any trigger output. etc.
//declare your other pins here for your next buttons

SimpleTimer timer;           // add simple timer for the time interval scanning period of phy
WidgetLED led1(V0);

void checkPhysicalButton();
int btnPin1State = LOW;      // ON
int RelayPin1State = HIGH;   // OFF
```

```

//*****Sets Relays to Off Position*****
#define TURN_ON 0           // TURN_ON and TURN_OFF are defined to account for Active High
#define TURN_OFF 1          // Used to switch relay states

void setup()
{
  Serial.begin (9600);
  Blynk.begin(auth, ssid, pass);
  {
    pinMode(RelayPin1, OUTPUT);           // initialize your pin as output.
    pinMode(btnPin1, INPUT_PULLUP);       // initialize your pin as input with enable internal
    digitalWrite(RelayPin1, RelayPin1State);
    digitalWrite(RelayPin1, TURN_OFF);    // remain off till command is receive

    // Setup a function to be called every 100 ms
    timer.setInterval(100L, checkPhysicalButton);
  }
  while (Blynk.connect() == false) {
    // Wait until connected
  }
}
void checkPin()
{
  // Invert state, since button is "Active LOW"
  if (digitalRead(btnPin1)) {
    led1.on();
  } else {
    led1.off();
  }
}
}
// Every time we connect to the cloud...
BLYNK_CONNECTED() {
  // Request the latest state from the server
  Blynk.syncVirtual(V1);
  // Alternatively, you could override server state using:
  //Blynk.virtualWrite(V1, RelayPin1State);
}
// When App button is pushed - switch the state
BLYNK_WRITE(V1) { // Map this Virtual Pin to your Mobile Blynk apps w
  RelayPin1State = param.asInt();
  digitalWrite(RelayPin1, RelayPin1State);
  {
    //BLYNK_LOG("Got a value: %s", param.asStr());
    // You can also use:
    int i = param.asInt();
    int state;
    // Switch mode inverse//
    if (i == 0) {
      digitalWrite(RelayPin1, HIGH);
      Serial.println("Relay 1 = OFF"); // turn OFF your relay outputs
    }
    if (i == 1) {
      digitalWrite(RelayPin1, LOW); // turn ON your relay outputs
      Serial.println("Relay 1 = ON");
    }
  }
}

```



```

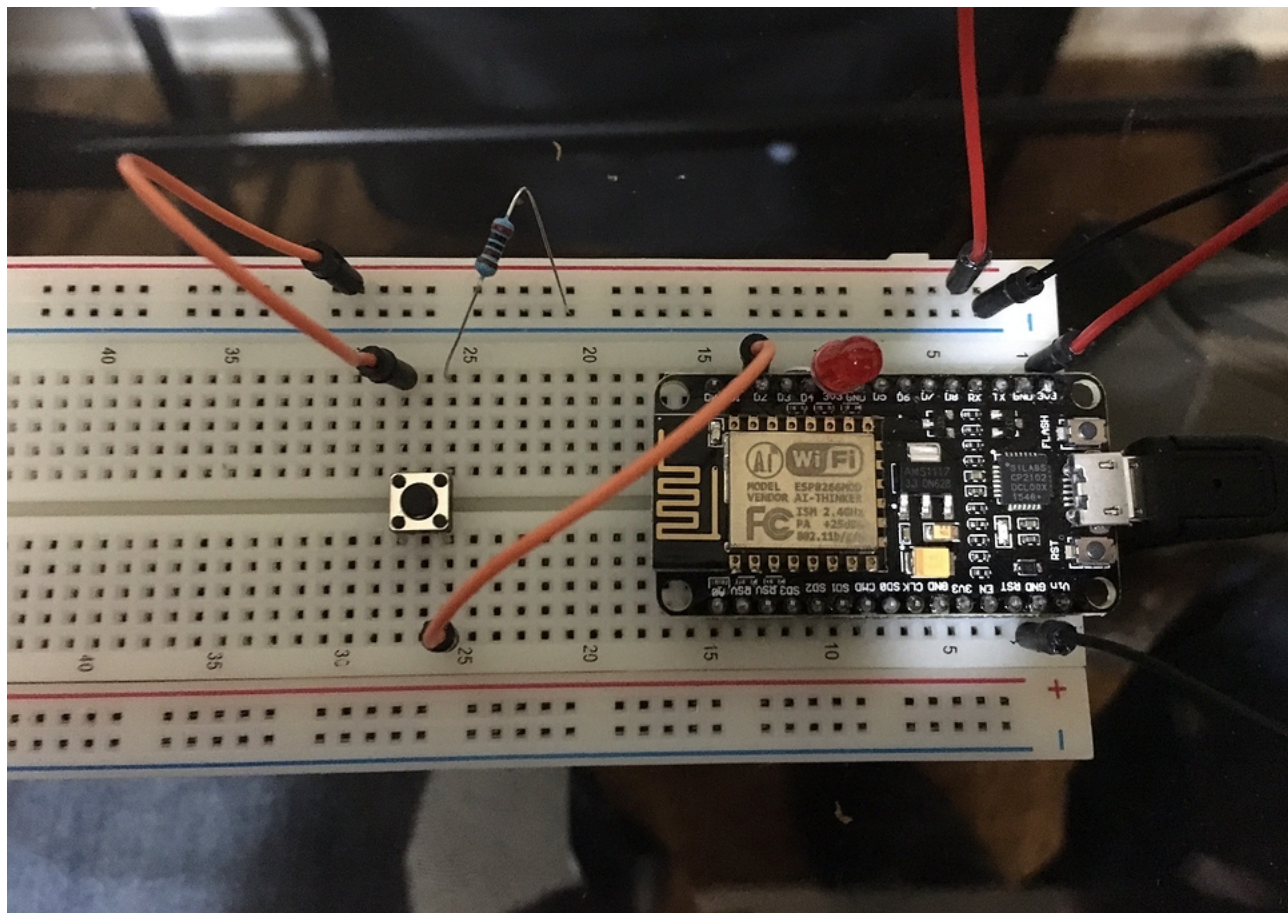
        delay (300);                // Delay response of your button switch (change it if
    }
}
}

void checkPhysicalButton()
{
    if (digitalRead(btnPin1) == LOW) {
        // btnPin1State is used to avoid sequential toggles
        if (btnPin1State != LOW) {
            // Toggle LED state
            RelayPin1State = !RelayPin1State;
            digitalWrite(RelayPin1, RelayPin1State);
            // Update Button Widget
            Blynk.virtualWrite(V1, RelayPin1State);
        }
        btnPin1State = LOW;
    } else {
        btnPin1State = HIGH;
    }
}

void loop()
{
    Blynk.run();                    // Run Blynk
    timer.run();                    // Run SimpleTimer
}

```

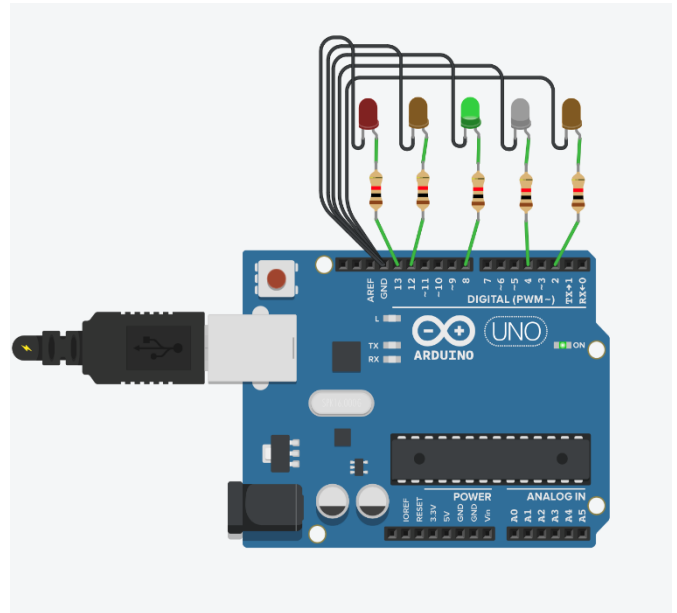
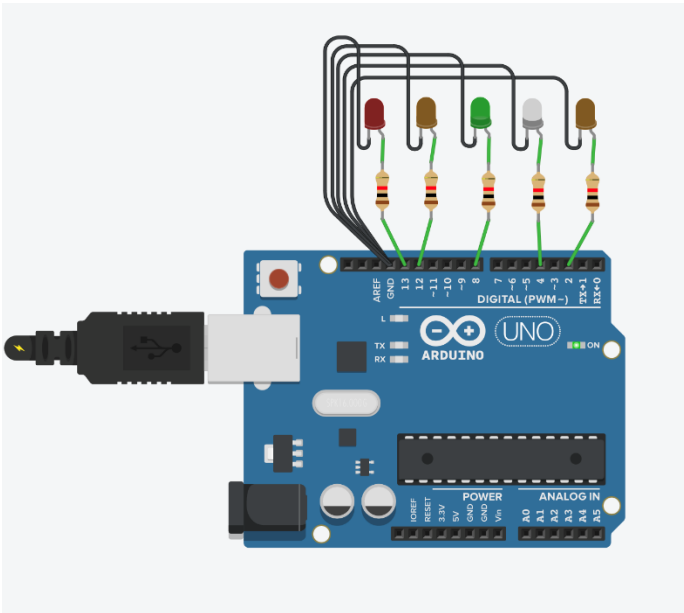
OUTPUT:



Q- Write a program to blink 5 different lights using array.

```
1 //5 different color LED blink using Arrays
2 int timer = 500;
3 int LEDPins[] = {13, 12, 8, 4, 2};          // an array of declared pin numbers on the board
4 int countOfpin = 5;                        // the number of pins, corrected to match the size of LEDPins array
5 void setup()
6 {
7     // we have declared an array to initialize the LED pins as OUTPUT
8     for (int PIN = 0; PIN < countOfpin; PIN++)
9     {
10         pinMode(LEDPins[PIN], OUTPUT);
11     }
12 }
13 void loop()
14 {
15     // loop starting from the lowest pin in the array to the highest:
16     for (int PIN = 0; PIN < countOfpin; PIN++) {
17         // turns the pin ON:
18         digitalWrite(LEDPins[PIN], HIGH);
19         delay(timer);
20         // turns the pin OFF:
21         digitalWrite(LEDPins[PIN], LOW);
22     }
23     // loop from the highest pin in the array to the lowest:
24     // It means the LEDs will light in the reverse direction as used above
25     for (int PIN = countOfpin - 1; PIN >= 0; PIN--)
26     {
27         digitalWrite(LEDPins[PIN], HIGH);
28         delay(timer);
29         digitalWrite(LEDPins[PIN], LOW);
30     }
31 }
32
```

OUTPUT:



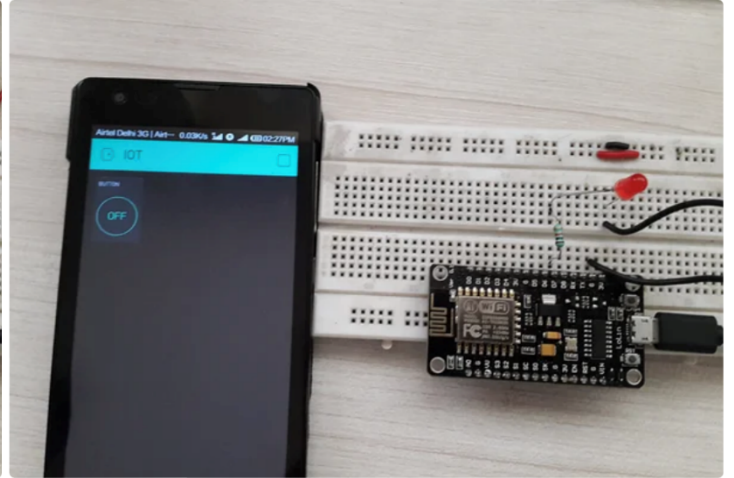
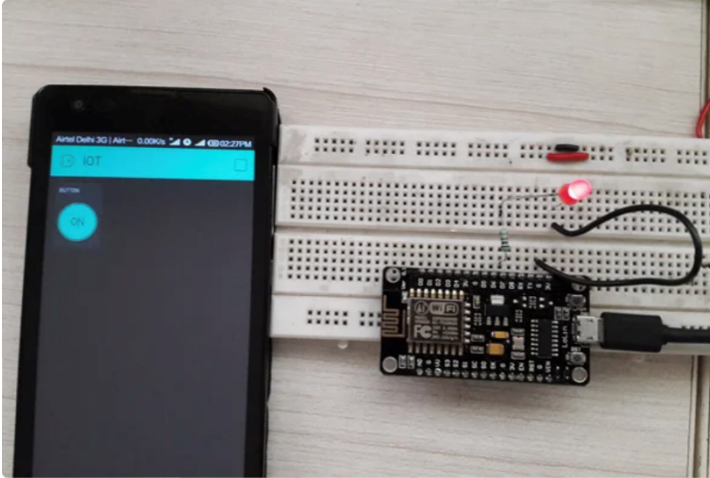
Q- Write a program to fade LED using NodeMCU(ESP8266) and blynk app.

II. CODE

```
#include <ESP8266WiFi.h>
#define ledPin 2
int brightness = 0;
int fadeAmount = 1;
void setup() {
    Serial.begin(115200);
    pinMode(ledPin,OUTPUT);
}
void loop() {
    Serial.println("10% Duty Cycle");
    analogWrite(ledPin,102);
    delay(2000);
    Serial.println("20% Duty Cycle");
    analogWrite(ledPin,205);
    delay(2000);
    Serial.println("50% Duty Cycle");

    analogWrite(ledPin,511);
    delay(2000);
    Serial.println("70% Duty Cycle");
    analogWrite(ledPin,714);
    delay(2000);
    Serial.println("100% Duty Cycle");
    analogWrite(ledPin,1023);
    delay(2000);
    Serial.println("LED Fadding begins.....");
    while(1){
        analogWrite(ledPin, brightness);
        brightness = brightness + fadeAmount;
        if (brightness <= 0 || brightness >= 1023) {
            fadeAmount = -fadeAmount;
        }
        delay(10);
    }
}
```

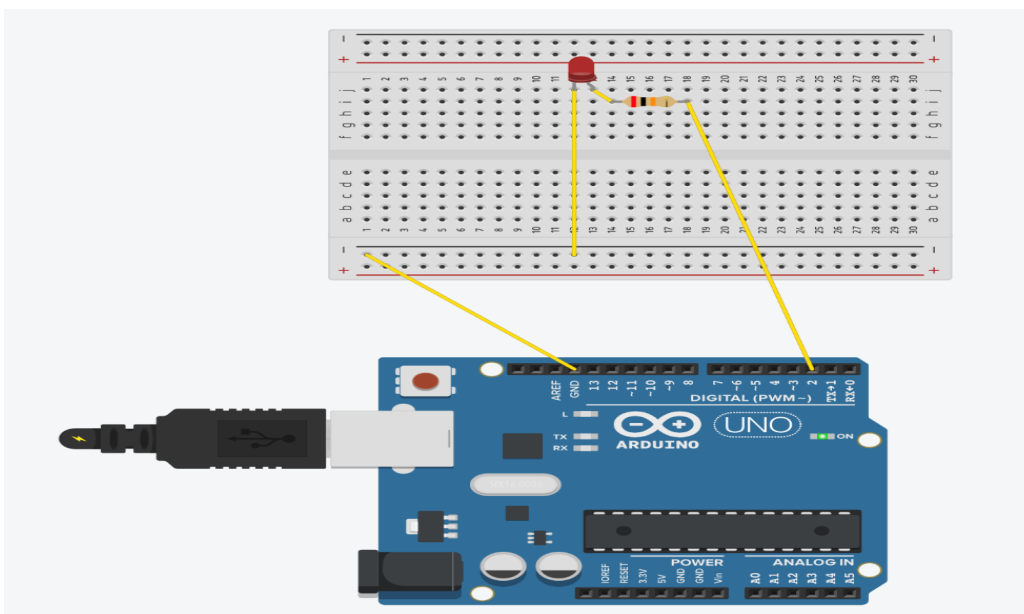
OUTPUT:



Q- Write a program in Arduino for Blink LED.

```
1  int ledPin=2;
2  void setup()
3  {
4      pinMode(ledPin,OUTPUT);
5  }
6  void loop()
7  {
8      digitalWrite(ledPin, HIGH);
9      delay(1000);
10     digitalWrite(ledPin, LOW);
11     delay(1000);
12 }
```

OUTPUT:



Q- Write a program for Controlling Raspberry Pi with WhatsApp.

```
import pyautogui as pygu #To move mouse cursor and make click and keyboard strokes
from time import sleep #for delay
import pyperclip #to copy and past data
import webbrowser #to open webbrowser
import os #to close webbrowser
import RPi.GPIO as IO      # calling header file for GPIO's of PI
import time                # calling for time to provide delays in program

IO.setmode (IO.BOARD)      # programming the GPIO by BOARD pin numbers, GPIO21 is called as PIN40
IO.setup(40,IO.OUT)        # initialize digital pin40 as an output.

#Open the default webbrowser and open web.whastapp
webbrowser.open_new('https://web.whatsapp.com/')

default_message = [
    "Hi I am your Whatsapp Bot :robot \n from RaspberryPi. I can help you with basic home automatio
    "*turn on light* - _Turns on the led connected to pi_",
    "*turn off light* - _Turns off the led connected to pi_"

]

turn_on_light = [
    "Sure, your :bulb \n Light is now turned on"
]

turn_off_light = [
    "Okay, Your LED is not turned off"
]

#Wait for whatsapp page to
def open_whatsapp():
    # check if whatsapp opened successfully
    find_whatsapp_header = None
    while find_whatsapp_header is None:
        find_whatsapp_header = pygu.locateOnScreen("Whatsapp_header.JPG", confidence=.8)
        use_here_button_pos = pygu.locateOnScreen("use_here_button.JPG", confidence=.8)
        if (use_here_button_pos):
            print("Whatsapp is being used somewhere else, clicking on use here")
            sleep(2)
            pygu.moveTo(use_here_button_pos[0], use_here_button_pos[1], duration=0.5)
```

```

        pygu.click()
        print(".")
        sleep(2)
        return 1

#checks for new message and opens it
def new_chat_available():
    # Check for new messages
    green_circle_pos = pygu.locateOnScreen("green_circle.JPG", confidence=.8)
    if (green_circle_pos):
        sleep(2)
        pygu.moveTo(green_circle_pos[0], green_circle_pos[1], duration=0.5)
        pygu.click()
        sleep(1)
        ok_button_pos = pygu.locateOnScreen("ok_button.JPG", confidence=.8)
        if (ok_button_pos):
            pygu.moveTo(ok_button_pos[0], ok_button_pos[1], duration=0.5)
            pygu.click()

        return 1
    else:
        sleep(1)
        return 0

def read_last_message():
    smily_paperclip_pos = pygu.locateOnScreen("smily_paperclip.JPG", confidence=.6)
    pygu.moveTo(smily_paperclip_pos[0], smily_paperclip_pos[1])
    pygu.moveTo(smily_paperclip_pos[0] + 50, smily_paperclip_pos[1] - 35, duration=0.5)
    sleep(1)
    pygu.tripleClick()
    pygu.hotkey('ctrl', 'c')
    sleep(0.1)
    return (pyperclip.paste())

def get_response(incoming_message):
    if "CD_bot" in incoming_message:
        return default_message
    if "turn on light" in incoming_message:
        IO.output(40,1) # turn the LED on
        return turn_on_light
    if "turn off light" in incoming_message:
        IO.output(40,0) # turn the LED off
        return turn_off_light
    else:
        return ""

def send_message(message_content):
    for content in message_content:
        pygu.typewrite(content, interval=.02)
        pygu.hotkey('shift', 'enter')
    sleep(1)
    pygu.hotkey('enter') #Enter key to send the message

def new_message_available():
    current_mouse_pos = pygu.position()
    pointer_color = pygu.pixel(current_mouse_pos[0], current_mouse_pos[1])
    if (pointer_color == (255, 255, 255)):
        return 1
    else:
        return 0

if (open_whatsapp()): #if whatsapp page is opened successfully
    print("##Whatsapp page ready for automation##")

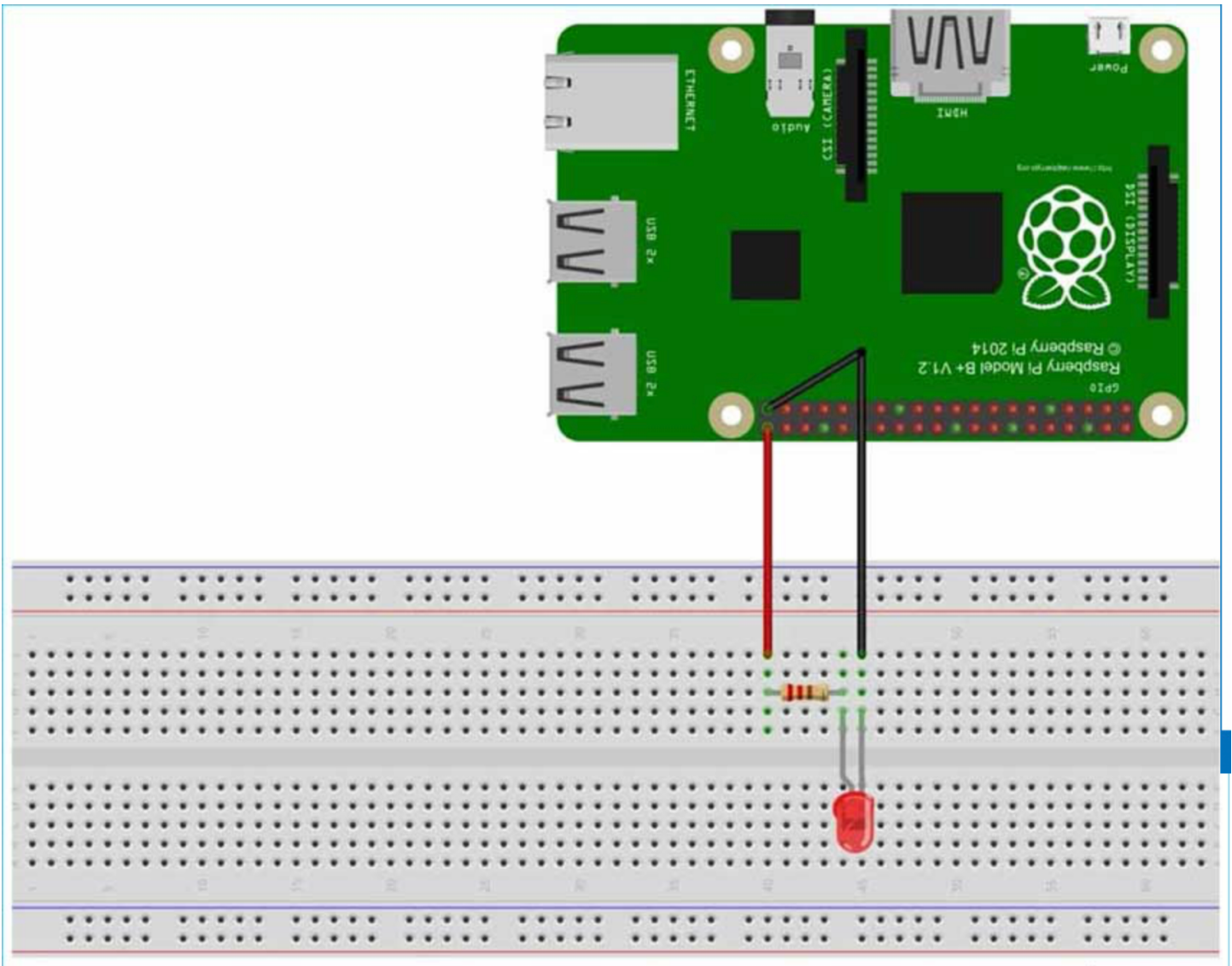
while(1):

    if (new_chat_available() or new_message_available()):
        print("New chat or message is available")
        incoming_message = read_last_message() #read the last message that we received

        message_content = get_response(incoming_message) #decide what to respond to that message
        send_message(message_content) #send the message to person

```


OUTPUT:



Q- Write a Program to perform sensor reading.

```

1  #define ECHOpin 5 // it defines the ECHO pin of the sensor to pin 5 of Arduino
2  #define TRIGpin 6
3  // we have defined the variable
4  long duration; // variable for the duration of sound wave travel
5  int distance; // variable for the distance measurement
6  void setup()
7  {
8      pinMode(TRIGpin, OUTPUT); // It sets the ECHO pin as OUTPUT
9      pinMode(ECHOpin, INPUT); // It sets the TRIG pin as INPUT
10     Serial.begin(9600); // // Serial Communication at the rate of 9600 bps
11     Serial.println("Test of the Ultrasonic Sensor HC-SR04"); // It will appear on Serial Monitor
12     Serial.println("with the Arduino UNO R3 board");
13 }
14 void loop()
15 {
16     // It first sets the TRIG pin at LOW for 2 microseconds
17     digitalWrite(TRIGpin, LOW);
18     delayMicroseconds(4);
19     // It now sets TRIG pin at HIGH for 15 microseconds
20     digitalWrite(TRIGpin, HIGH);
21     delayMicroseconds(15);
22     digitalWrite(TRIGpin, LOW);
23     // It will read the ECHO pin and will return the time
24     duration = pulseIn(ECHOpin, HIGH);
25     // distance formula
26     distance = duration*(0.034/2);
27     Serial.print("Distance: ");
28     Serial.print(distance);
29     Serial.println(" cm"); //specified unit of distance
30 }

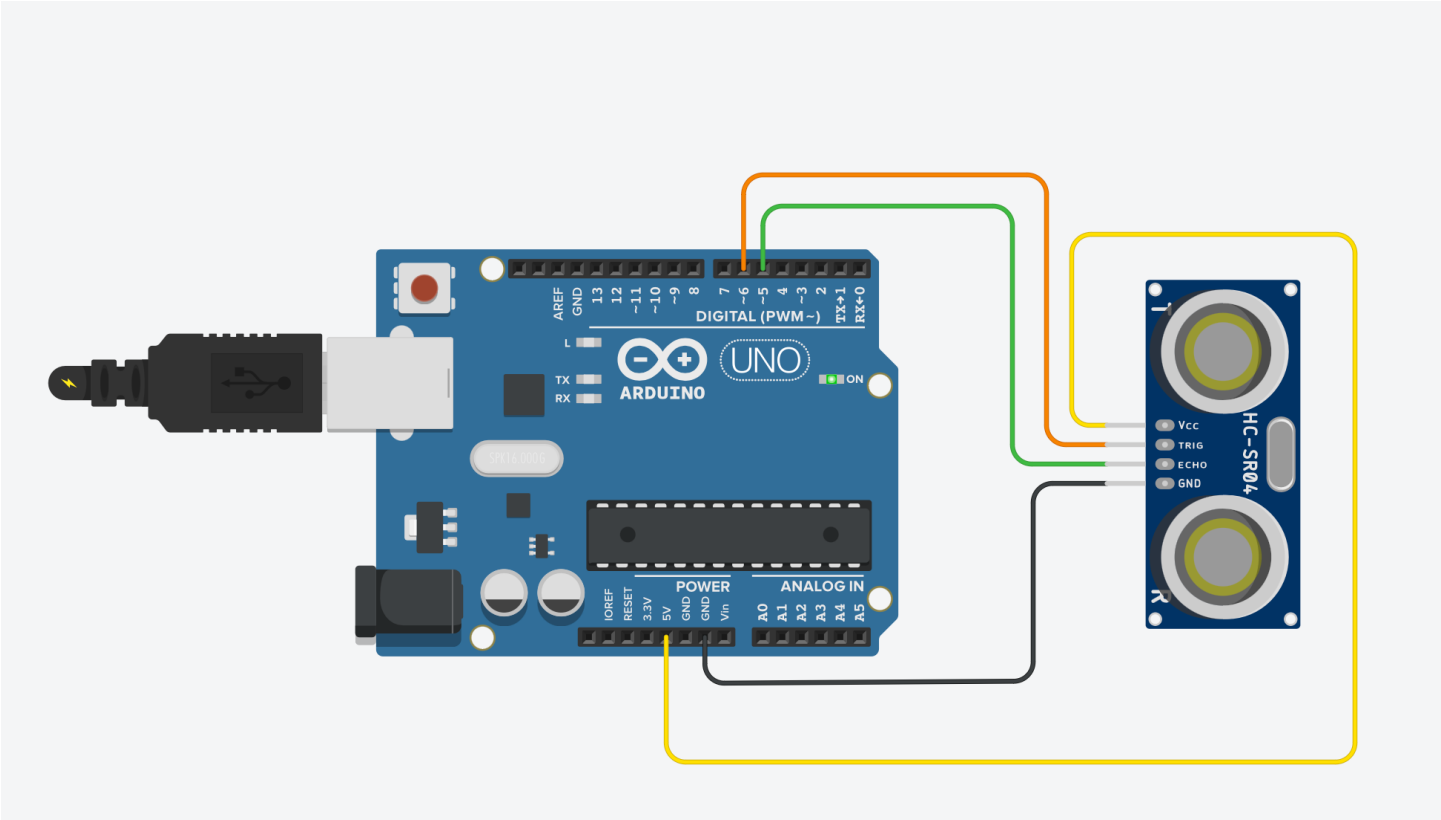
```

Output Serial Monitor ×

```
Message (Enter to send message to 'Arduino Uno' on '/dev/cu.usbmodem2017_2_251')
```

[illegible]

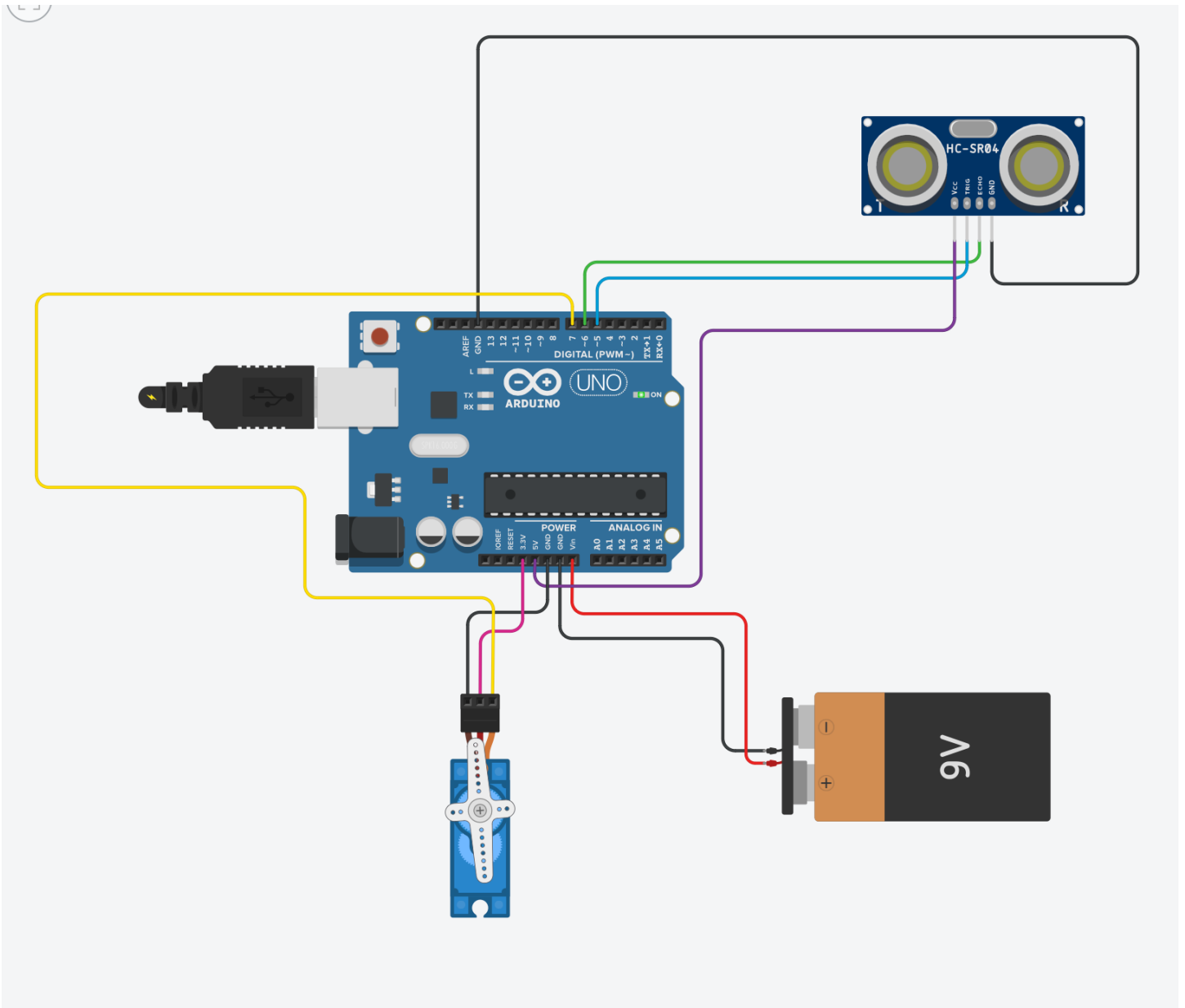
OUTPUT:



Q- Write a program for Arduino by using Ultrasonic Sensors and Servo motor (HC-SR04), and make a smart dustbin.

```
1  #include <Servo.h>          //servo library
2  Servo servo;
3  int trigPin = 5;
4  int echoPin = 6;
5  int servoPin = 7;
6  int led= 10;
7  long duration, dist, average;
8  long aver[3];    //array for average
9
10
11 void setup() {
12     Serial.begin(9600);
13     servo.attach(servoPin);
14     pinMode(trigPin, OUTPUT);
15     pinMode(echoPin, INPUT);
16     servo.write(0);          //close cap on power on
17     delay(100);
18     servo.detach();
19 }
20
21 void measure() {
22     digitalWrite(10,HIGH);
23     digitalWrite(trigPin, LOW);
24     delayMicroseconds(5);
25     digitalWrite(trigPin, HIGH);
26     delayMicroseconds(15);
27     digitalWrite(trigPin, LOW);
28     pinMode(echoPin, INPUT);
29     duration = pulseIn(echoPin, HIGH);
30     dist = (duration/2) / 29.1;    //obtain distance
31 }
32 void loop() {
33     for (int i=0;i<=2;i++) {    //average distance
34         measure();
35         aver[i]=dist;
36         delay(10);              //delay between measurements
37     }
38     dist=(aver[0]+aver[1]+aver[2])/3;
39
40     if ( dist<50 ) {
41         //Change distance as per your need
42         servo.attach(servoPin);
43
44         delay(1);
45         servo.write(0);
46         delay(3000);
47         servo.write(150);
48         delay(1000);
49         servo.detach();
50     }
51     Serial.print(dist);
52 }
```

OUTPUT:



Q- Write a program for monitoring temperature using Arduino and LM35 Temperature Sensors.

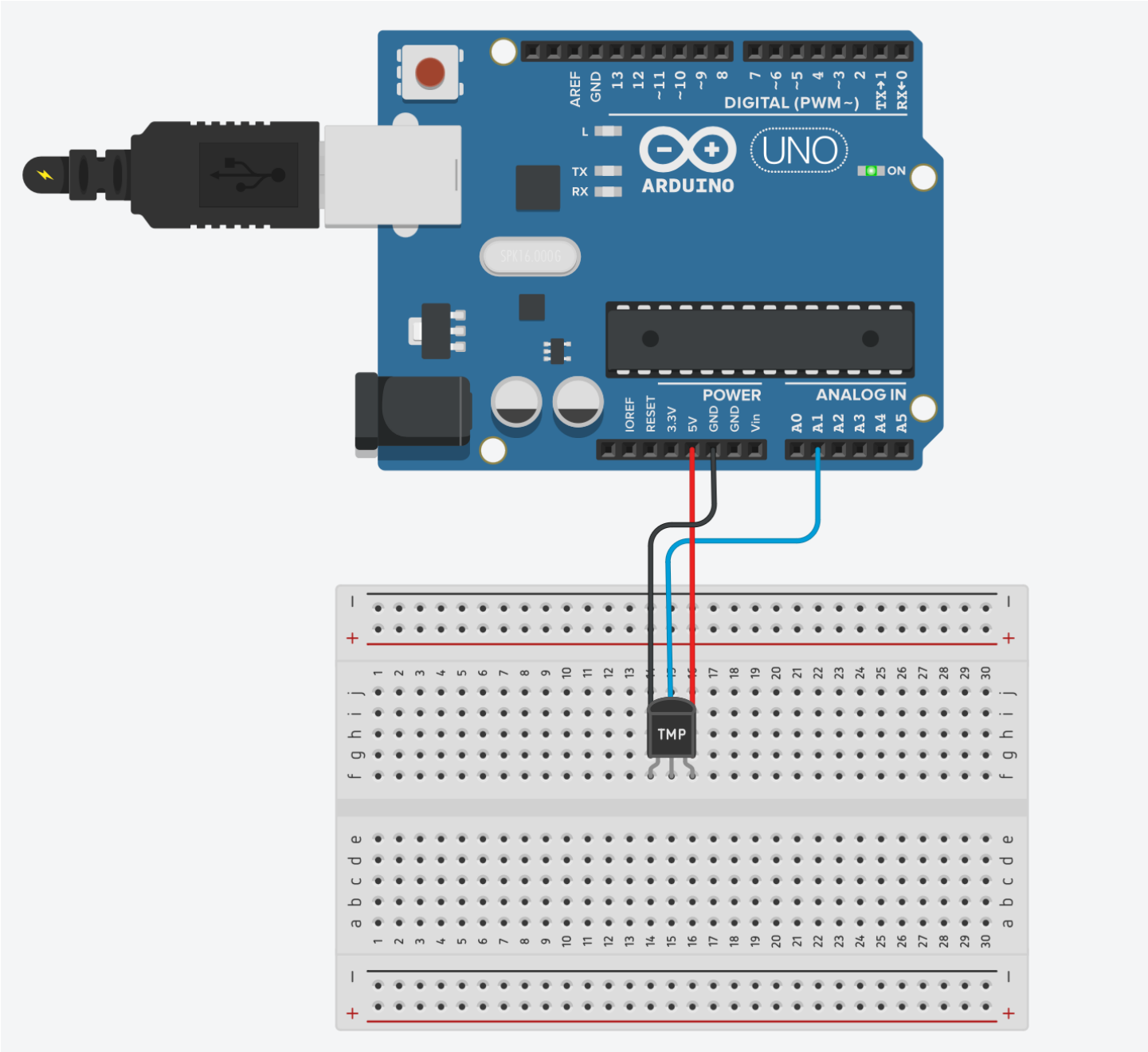
```
1  const int lm35_pin = A1;      /* LM35 O/P pin */
2
3  void setup() {
4      Serial.begin(9600);
5  }
6
7  void loop() {
8      int temp_adc_val;
9      float temp_val;
10     temp_adc_val = analogRead(lm35_pin); /* Read Temperature */
11     temp_val = (temp_adc_val * 4.88); /* Convert adc value to equivalent temperature */
12     temp_val = (temp_val/10); /* LM35 gives output of 10mv/°C */
13     Serial.print("Temperature = ");
14     Serial.print(temp_val);
15     Serial.print(" Degree Celsius\n");
16     delay(1000);
17 }
```



Serial Monitor

```
Temperature = 499.22 Degree Celsius
Temperature = 499.22 Degree Celsius
Temperature = 499.22 Degree Celsius
Temperature = 499.22 Degree Celsius
Temperature = 499.22 Degree Celsius
Temperature = 499.22 Degree Celsius
```

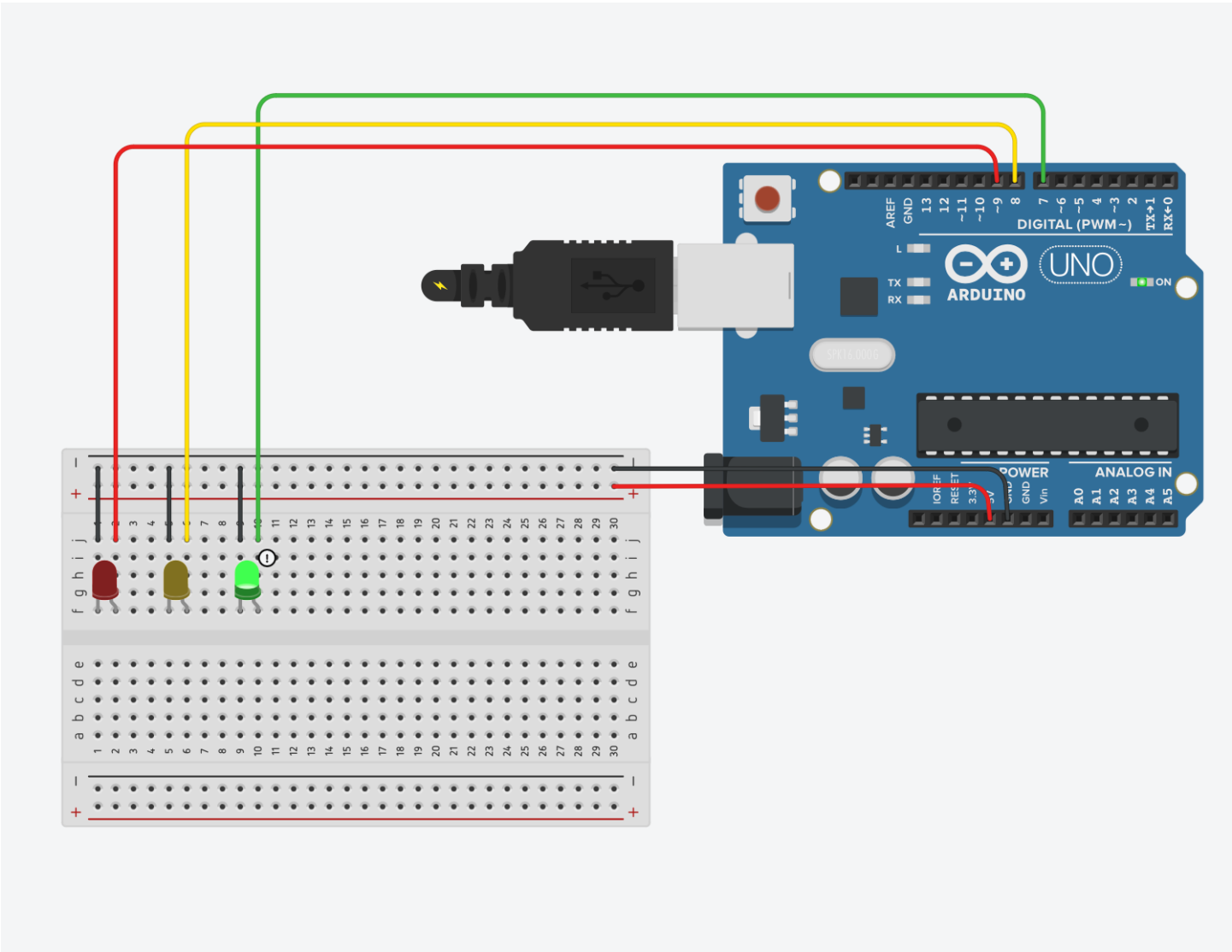
OUTPUT:



Q- Write a program to make a traffic light using 3 LEDs.

```
1  //Code for Traffic Light Projection using 3 Different color LEDs.
2  int red = 9;
3  int yellow = 8;
4  int green = 7;
5
6  void setup() {
7      pinMode(red, OUTPUT);
8      pinMode(yellow, OUTPUT);
9      pinMode(green, OUTPUT);
10 }
11
12 void loop() {
13
14     // Turn on the yellow LED
15     digitalWrite(yellow, HIGH);
16     delay(5000); // Wait for 5 seconds
17     digitalWrite(yellow, LOW);
18
19     // Turn on the green LED
20     digitalWrite(green, HIGH);
21     delay(20000); // Wait for 20 seconds
22     digitalWrite(green, LOW);
23
24     // Turn on the red LED
25     digitalWrite(red, HIGH);
26     delay(20000); // Wait for 20 seconds
27     digitalWrite(red, LOW);
28
29 }
```


OUTPUT:



Q- Write a program to show how to fade an LED on pin 9 using the Analog Write () function.

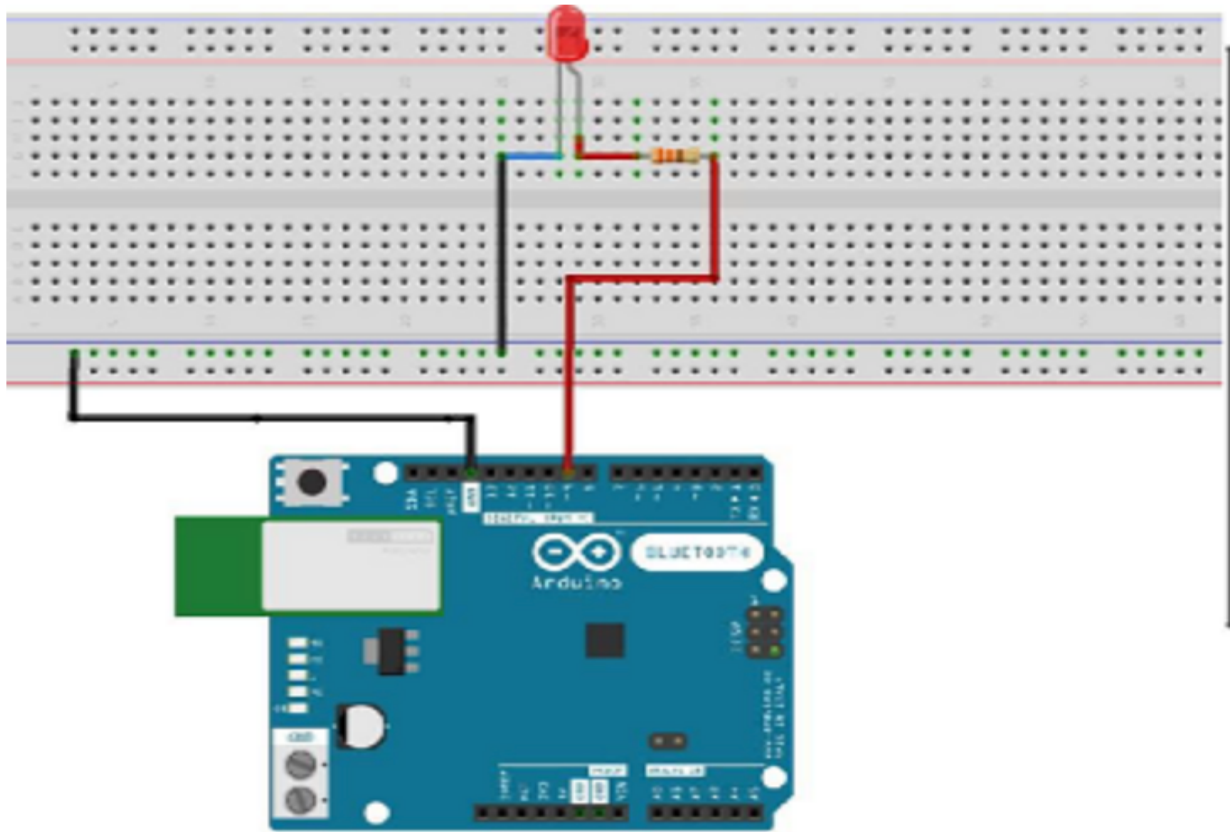
```
int led = 9; // the PWM pin the LED is attached to
int brightness = 0; // how bright the LED is
int fadeAmount = 5; // how many points to fade the LED by
// the setup routine runs once when you press reset:

void setup() {
  // declare pin 9 to be an output:
  pinMode(led, OUTPUT);
}

// the loop routine runs over and over again forever:

void loop() {
  // set the brightness of pin 9:
  analogWrite(led, brightness);
  // change the brightness for next time through the loop:
  brightness = brightness + fadeAmount;
  // reverse the direction of the fading at the ends of the fade:
  if (brightness == 0 || brightness == 255) {
    fadeAmount = -fadeAmount ;
  }
  // wait for 30 milliseconds to see the dimming effect
  delay(300);
}
```

OUTPUT:



Q- Write the Steps to add libraries in Arduino and setup of Arduino IDE for Programming.

Code//

Using the Library manager

1. Open the Library Manager:

- In the menu bar, select *Tools > Manage Libraries...*
- In IDE 2, you can also click on the  button in the sidebar.

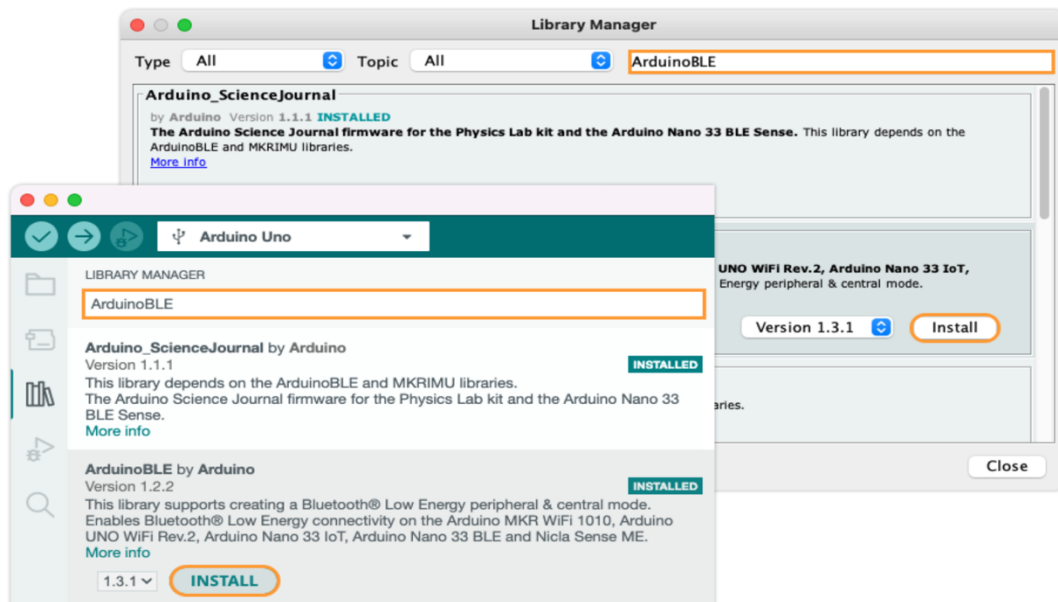
2. Filter the available libraries by typing something (such as a library name) in the text field above the listed libraries.

IDE 1.x also has options to filter by *Type* and *Topic*.

3. Find the library in the search results. The results are listed alphabetically, so you may need to scroll down the list.

To find more information about the library, click **More info**. This will usually take you to a reference page or repository for the library

4. Find a library you want to install. You can review the description and author. When you've found a library you want to install, click **Install**. The latest version is selected by default.



The Library Manager looks slightly different depending on what version of the IDE you are using.

5. Wait for the installation to complete.

To find more information about the library, click **More info**. This will usually take you to a reference page or repository for the library

Importing a .zip Library

1. In the menu bar, go to `Sketch > Include Library > Add .ZIP Library...`
2. You will be prompted to select the library you want to add. Navigate to the .zip file's location and open it.
3. If you're using Arduino IDE 2, you may need to restart it for the library to be available^[1].

Manual installation

1. Go to the directory where you have downloaded the .zip library file.
2. Extract the .zip file.
3. Select the main folder (it should have the library's name) and move it to the "libraries" folder [inside your sketchbook](#).
4. Restart Arduino IDE.

Using installed libraries

When the library has been installed you can do the following:

- Open any included example sketches from *File > Examples > <Library Name>* in the top menu bar. Note that all libraries do not come with examples.
- Include the library in your sketch by selecting *Sketch > Include Library > <Library Name>*.

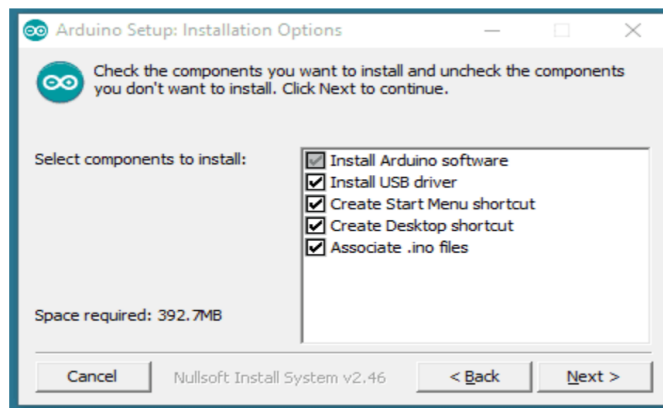
Q- Study and install IDE of Arduino.

Code

Windows

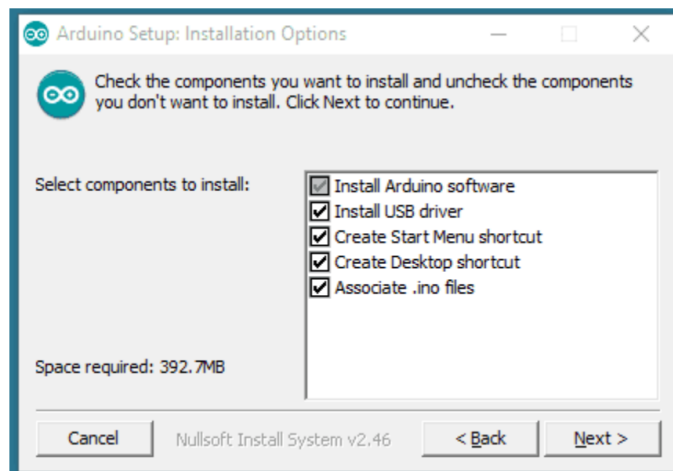
1. [Download the latest release](#) (The download will start after you click this link. Check your browser's download manager or the Downloads folder on your computer to find the downloaded file)
2. Double-click the executable (.exe) file.
3. Follow the instructions in the installation guide.
4. When completing the setup, leave *Run Arduino IDE* ticked to launch the application, or launch it later from the Start Menu.

When the download finishes, proceed with the installation and please allow the driver installation process when you get a warning from the operating system.

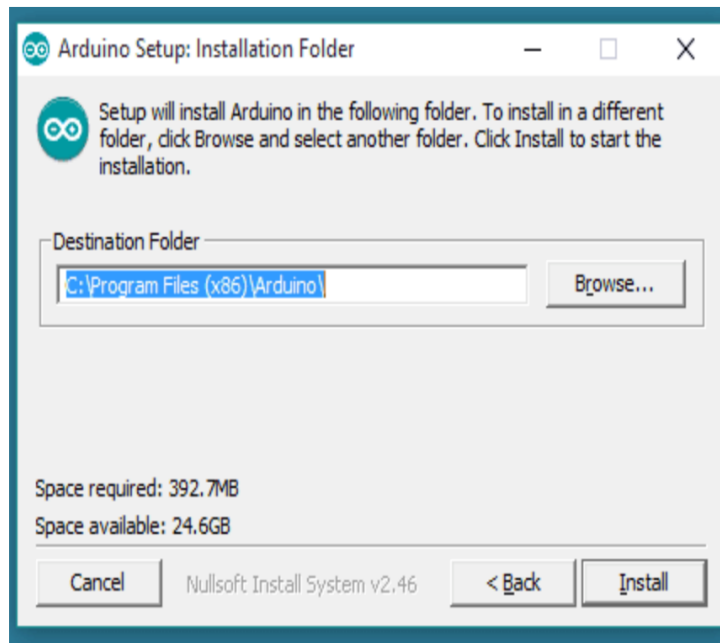


Choose the components to install.

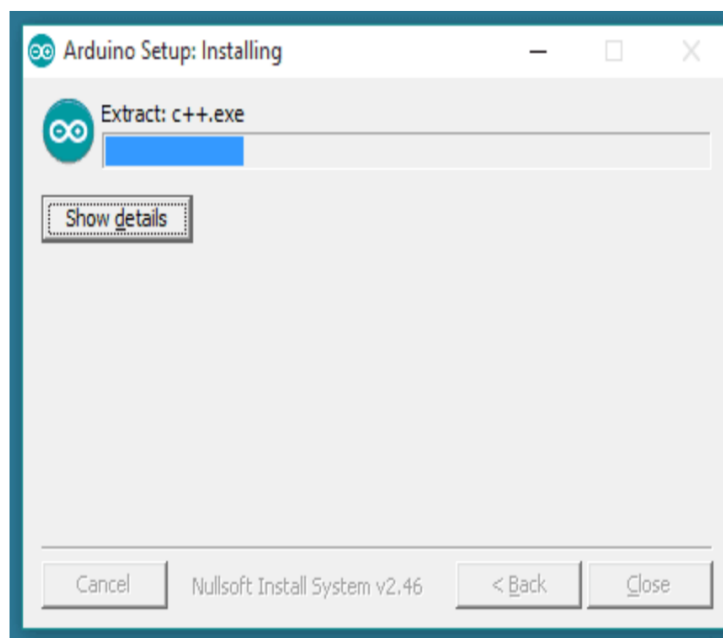
When the download finishes, proceed with the installation and please allow the driver installation process when you get a warning from the operating system.



Choose the components to install.



Choose the installation directory.



Installation in progress.

Q- Write the steps to add blynk libraries for NodeMCU and account on IFTTT for home automation.

Code

1. Install Blynk Libraries for NodeMCU:

- Open your Arduino IDE.
- Go to **Sketch > Include Library > Manage Libraries**.
- Search for “blynk” and install the Blynk library.
- If you’re using ESP8266 or NodeMCU for the first time, you’ll also need to add NodeMCU to the boards manager and install the `ESP8266WiFi.h` library [1](#) [2](#).

2. Configure the Blynk App:

- Download the Blynk App from Google Play Store or the App Store.
- Create a new project in the Blynk app:
 - Enter the project name.
 - Choose the device (in this case, select NodeMCU).
- Blynk will send an **Auth Token** to your registered email address. You’ll need this token while programming the ESP8266.

3. Adding Widgets in Blynk App:

- Open your Blynk project in the app.
- Click the “+” icon at the top.
- Select **Button**.
- Configure the buttons for controlling the 4-channel relay module:
 - For each button, choose the output pin (e.g., V1, V2, V3, V4) and set the mode to **Switch**.
 - Note: If you’re using an active low relay module, send “0” to turn ON the relay and “1” to turn OFF the relay [3](#).

4. Program NodeMCU with Arduino IDE:

- Use the Arduino IDE to program NodeMCU:
 - Update the Preferences > Additional Boards Manager URLs with the following URLs:
 - [ESP32 Boards Manager URL](#)
 - [ESP8266 Boards Manager URL](#)
 - Write the code for your Blynk home automation project.
 - Upload the code to your NodeMCU.

5. Connect Home Appliances with Relay:

- Set up the circuit:
 - Use an active low 5V relay module.
 - Instead of connecting pull-up resistors with the push buttons, use the `INPUT_PULLUP` function in your code.
 - Power the circuit using a 5V mobile charger.
- Connect the relay module to your home appliances.

6. IFTTT Account Setup:

- Go to the IFTTT website and create an account.
- Explore existing applets or create your own:
 - Connect Blynk and IFTTT services.
 - Define triggers (e.g., button press in Blynk) and actions (e.g., turn on a smart bulb via IFTTT).